

Отчёт по лабораторной работе №23

по курсу «Алгоритмы и структуры данных».

Выполнил студент группы М8О-114БВ-25 Богатырева Екатерина Денисовна № по списку 2.

Контакты: bogatyrevaK2019@yandex.ru

Работа выполнена: «30» марта 2026 г.

Преподаватель: каф.806 Сысоев Максим Алексеевич

Входной контроль знаний с оценкой: _____

Отчет сдан «09» апреля 2026 г.

Итоговая оценка: _____

Подпись преподавателя: _____

1. **Тема:** Динамические структуры данных. Обработка деревьев.
2. **Цель работы:** Составить программу на языке Си для построения и обработки упорядоченного двоичного дерева, содержащего узлы типа `enum`. Реализовать основные функции работы с деревом (добавление, удаление, визуализация) и вычислить функцию в соответствии с вариантом.
3. **Задание (Вариант № 2):** Проверить, является ли двоичное дерево В-деревом.
4. **Оборудование:** *Оборудование ПЭВМ студента, если использовалось:*
Процессор: Intel(R) Pentium(R) CPU 6405U @ 2.40GHz 2.40 GHz, ОП 8 Гб, SSD 256 Гб
5. **Программное обеспечение:** *Программное обеспечение ЭВМ студента, если использовалось:*
Операционная система семейства Unix, наименование Ubuntu, версия 24.04 LTS, интерпретатор команд `bash` версия 5.2.21.
Система программирования: gcc (Ubuntu 11.4.0-1ubuntu1~22.04.2) версия 11.4 Редактор текстов: VS code версия: 1.107.1.
Язык программирования: Си
6. **Идея, метод, алгоритм решения задачи [в формах: словесной, псевдокода, графической (блок-схема, диаграмма, рисунок, таблица) или формальные спецификации с пред- и постусловиями]:**
Идея: В-дерево — это двоичное дерево, в котором нет узлов с одним потомком. Каждый узел либо лист (0 детей), либо имеет ровно двух детей. Таким образом, для проверки достаточно рекурсивно обойти дерево и убедиться, что ни у одного узла количество дочерних узлов не равно 1.

Метод:

- Дерево строится как упорядоченное (бинарное поиска) на основе перечисления стран.
- Реализованы стандартные функции: добавление, удаление, визуализация.
- Для варианта 2 реализована функция `isBTree(Node* root)`, которая рекурсивно проверяет:
 - Если узел `NULL` → `true`
 - Если узел имеет ровно одного потомка → `false`
 - Иначе рекурсивно проверить левое и правое поддеревья.

Алгоритм:

Начало

если `root == NULL` → вернуть `true`

подсчитать количество ненулевых потомков (`left/right`)

если ровно один потомок → вернуть `false`

иначе → вернуть (`isBTree(left)` и `isBTree(right)`)

7. **Сценарий выполнения работы** (план работы, первоначальный текст программы в черновике (можно на отдельном листе) и тесты либо соображения по тестированию).

План работы:

1. Изучить свойства В-дерева и бинарного дерева поиска.
2. Спроектировать структуру узла с данными типа enum Country.
3. Реализовать базовые функции: createNode, addNode, deleteNode, printTree, freeTree.
4. Реализовать функцию проверки на В-дерево isBTree.
5. Создать текстовое меню.
6. Протестировать на различных конфигурациях дерева.

Тесты:

МЕНЮ

- 1 - Добавить страну
- 2 - Показать дерево
- 3 - Удалить страну
- 4 - Проверить, В-дерево ли это
- 5 - Выход

Выполнить функцию: 4

Да: дерево является В-деревом

Соображения по тестированию: Необходимо проверить корректность вставки узлов с построением различных конфигураций дерева (левая ветка, правая ветка, сбалансированное дерево). Важно протестировать **функцию** isBTree в следующих ситуациях:

- Пустое дерево (корень = NULL) — должно возвращать true, так как нет узлов степени 1.
- Дерево из одного узла — возвращает true (узел степени 0).
- Дерево, где все узлы имеют 0 или 2 потомков — true.
- Дерево с любым узлом степени 1 (только левый или только правый потомок) — false.
- Дерево с узлом степени 1 на разных уровнях (корень, середина, предлист) — false.
- После удаления узлов — проверка корректного изменения статуса В-дерева.
- Визуализация дерева — корректность отображения уровней вложенности.
- Устойчивость к некорректному вводу (буквы вместо цифр, номера вне диапазона 1-7).

Пункты 1-7 отчета составляются строго до начала лабораторной работы.

Допущен к выполнению работы. Подпись преподавателя _____

8. **Распечатка протокола** (подклеить листинг окончательного варианта программы с тестовыми примерами, подписанный преподавателем).

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <windows.h>
#include <stdbool.h>
```

```
typedef enum {
    RUSSIA = 1,
    USA,
    CHINA,
    GERMANY,
    FRANCE,
    JAPAN,
    UK
} Country;
```

```
typedef struct Node {
    Country data;
    struct Node *left;
    struct Node *right;
} Node;
```

```
const char* countryToString(Country c) {
```

```

switch(c) {
    case RUSSIA: return "Russia";
    case USA:    return "USA";
    case CHINA:  return "China";
    case GERMANY: return "Germany";
    case FRANCE: return "France";
    case JAPAN:  return "Japan";
    case UK:     return "UK";
    default:     return "???" ;
}
}

Node* createNode(Country value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) return NULL;
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

Node* addNode(Node* root, Country value) {
    if (root == NULL)
        return createNode(value);
    if (value < root->data)
        root->left = addNode(root->left, value);
    else if (value > root->data)
        root->right = addNode(root->right, value);
    else
        printf("Страна уже есть в дереве!\n");
    return root;
}

Node* findMin(Node* root) {
    while (root->left != NULL)
        root = root->left;
    return root;
}

Node* deleteNode(Node* root, Country value) {
    if (root == NULL) {
        printf("Страна не найдена!\n");
        return NULL;
    }
    if (value < root->data)
        root->left = deleteNode(root->left, value);
    else if (value > root->data)
        root->right = deleteNode(root->right, value);
    else {
        if (root->left == NULL) {
            Node* temp = root->right;
            free(root);
            return temp;
        }
        else if (root->right == NULL) {
            Node* temp = root->left;

```

```

        free(root);
        return temp;
    }
    Node* temp = findMin(root->right);
    root->data = temp->data;
    root->right = deleteNode(root->right, temp->data);
}
return root;
}

bool findValue(Node* root, Country value) {
    if (root == NULL) return false;
    if (root->data == value) return true;
    if (value < root->data)
        return findValue(root->left, value);
    return findValue(root->right, value);
}

void printTree(Node* root, int level) {
    if (root == NULL) return;
    printTree(root->right, level + 1);
    for (int i = 0; i < level; i++)
        printf(" ");
    printf("%s\n", countryToString(root->data));
    printTree(root->left, level + 1);
}

// Функция для варианта 2: проверка на В-дерево
bool isBTree(Node* root) {
    if (root == NULL) return true;
    int children = (root->left != NULL) + (root->right != NULL);
    if (children == 1) return false; // узел степени 1
    return isBTree(root->left) && isBTree(root->right);
}

void freeTree(Node* root) {
    if (root == NULL) return;
    freeTree(root->left);
    freeTree(root->right);
    free(root);
}

int main() {
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
    setlocale(LC_ALL, "ru_RU.UTF-8");

    Node* root = NULL;
    int choice, val;

    while (1) {
        printf("\nМЕНЮ\n");
        printf("1 - Добавить страну\n");
        printf("2 - Показать дерево\n");
        printf("3 - Удалить страну\n");
        printf("4 - Проверить, В-дерево ли это\n");
    }
}

```

```

printf("5 - Выход\n");
printf("Выбор: ");

if (scanf("%d", &choice) != 1) {
    printf("Ошибка ввода!\n");
    break;
}

switch (choice) {
    case 1:
        printf("\n1-Russia,2-USA,3-China,4-Germany,5-France,6-Japan,7-UK\n");
        printf("Введите номер страны: ");
        scanf("%d", &val);
        if (val < 1 || val > 7) {
            printf("Ошибка: номер от 1 до 7\n");
            break;
        }
        root = addNode(root, (Country)val);
        break;

    case 2:
        if (root == NULL)
            printf("Дерево пусто!\n");
        else
            printTree(root, 0);
        break;

    case 3:
        if (root == NULL) {
            printf("Дерево пусто!\n");
            break;
        }
        printf("Введите номер страны для удаления: ");
        scanf("%d", &val);
        if (val < 1 || val > 7) {
            printf("Ошибка: номер от 1 до 7\n");
            break;
        }
        if (!findValue(root, (Country)val))
            printf("Такой страны нет в дереве!\n");
        else
            root = deleteNode(root, (Country)val);
        break;

    case 4:
        if (isBTree(root))
            printf("Да: дерево является В-деревом\n");
        else
            printf("Нет: дерево не является В-деревом\n");
        break;

    case 5:
        freeTree(root);
        printf("Программа завершена.\n");
        return 0;
}

```

```

        default:
            printf("Неверный выбор! Введите 1-5\n");
        }
    }
    return 0;
}

```

9. **Дневник отладки** должен содержать дату и время сеансов отладки и основные события (ошибки в сценарии и программе, нестандартные ситуации) и краткие комментарии к ним. В дневнике отладки приводятся сведения об использовании других ЭВМ, существенном участии преподавателя и других лиц в написании и отладке программы.

№	Лаб. или дом.	Дата	Время	Событие	Действие по исправлению	Примечание

10. **Замечания автора** по существу работы: замечания отсутствуют.

11. **Выводы:** В ходе выполнения лабораторной работы была разработана программа на языке Си, реализующая упорядоченное двоичное дерево с узлами типа `enum`. Реализованы основные операции: добавление, удаление с сохранением упорядоченности, текстовая визуализация. Для варианта 2 реализована проверка, является ли дерево В-деревом (отсутствие узлов с одним потомком). Работа позволила закрепить навыки работы с динамическими структурами данных, рекурсивными алгоритмами и управлением памятью.

Подпись студента _____